

multiple nodes. So, the most difficult part is not about implementing the functions and writing code; it's about how you carve out and shape your data.

A (very) short tutorial

Hadoop is one of the most popular implementations (and it's open source, too) of this framework that was introduced by Google. There is also a project called Disco, which looks interesting to me, since the core of this project has been implemented in Erlang, along with some bits and pieces in Python. This should make it perform better than Hadoop, which is written in Java. Also, Hadoop tends to seem a bit more complex (and too massive for a framework, being a little more than 70 MB for the newer version!) than it should be for most users, so I suggest you take a look at the Disco project if you are a Python developer.

Even so, we will still examine Hadoop, since it is one of the most actively developed, right now. I will assume that you have already set up Hadoop, preferably version 0.21.0 in pseudo-distributed mode, if you don't have access to a cluster. Since the Hadoop website already has in-depth documentation for installing it, there is no point in repeating that here. Alternatively, I would recommend using Cloudera's distribution, which provides *deb* and *rpm* packages for easy installation on major Linux distributions. It even provides preconfigured VMWare images for download. Hadoop provides three basic ways of developing MapReduce applications:

- Java API
- Hadoop streaming
- Hadoop pipes

Java API

This is the native Java API that Hadoop provides, which is available as the *org.apache.hadoop.mapreduce* package in the new implementation, and the *org.apache.hadoop.mapred* as the older version. Instead of devoting a full-fledged step-by-step tutorial to developing a Java program with this API, let's just try to run some of the excellent examples already bundled with Hadoop. The example we are going to take up is the distributed version of the *Grep* command, which takes a text file as the input, and returns the number of occurrences of the specified word in the input file. You can take a look at the source code of the example, which can be found in the `HADOOP_HOME/mapred/src/examples/org/apache/examples/Grep.java` file, where `HADOOP_HOME` is a variable specifying the directory where you have installed Hadoop on your system. First of all, let's copy a text file to be searched, from the local filesystem, into HDFS, using the following command in Hadoop's home directory:

```
bin/hadoop fs -copyFromLocal path/on/local/filesystem /DFS/path/
```

```
to/text/file
```

Now that we have our inputs ready, we can run the MapReduce example using the following command:

```
bin/hadoop jar hadoop-mapred-examples-0.21.0.jar grep /DFS/path/
to/input/file /DFS/path/to/output/Directory keyword
```

Here, *keyword* is any word or regular expression that you are trying to find in the text file. After a lot of verbose text scrolling by, when your job is complete, you can take a look at the output:

```
bin/hadoop fs -cat /DFS/path/to/output/file
```

The output should look something like what's shown below, i.e., the keyword accompanied by the number of occurrences:

```
2      keyword
```

There are a number of other examples that are available, including the classic wordcount, which is almost synonymous with MapReduce tutorials on the Web. HBase also provides an API in Java for easier access to the HBase tables. These classes are available in the *org.apache.hadoop.hbase.mapreduce* package. An excellent tutorial on HBase and MapReduce can be found at <http://bit.ly/blj7IX>; go through it if you're interested.

Hadoop streaming

The Streaming utility allows you to program with any application by receiving input at the standard input and giving output at the standard output. Here, any executable can be specified as the mapper and reducer, as long as it is able to receive and send output from/to the standard interface. For example, the following command in Hadoop's home directory will send a MapReduce job for submission using the streaming utility:

```
bin/hadoop jar $HADOOP_HOME/mapred/contrib/
streaming/hadoop-0.21.0-streaming.jar
-input HDFSpathtoInputDirectory \
-output HDFSpathtoOutputDirectory \
-mapper mapperScript \
-reducer reducerScript \
-file AdditionalFile \
-file MoreAdditionalFiles
```

It is assumed that the mapper and reducer scripts are already present on all the nodes, if you are running a cluster, while the *file* argument allows you to specify any additional files that need to be sent to those nodes (they can even be mapper and reducer script files themselves, if